SENIOR FRONTEND ENGINEER

Transformation Roadmap

3 YOE → Senior-level thinking · 20 weeks · 15+ LPA

rds, snippet manager, real-time notifications, analytics dashboard,
t learned becomes a shipped feature.

React 19	TypeScript	Zustand	React Query	Vite
WebSockets	MSW	Playwright	Storybook	Recharts

20-Week Timeline at a Glance

Phase	Weeks	Focus	Key Deliverable
1 — Foundation	1–3	JS/TS mastery, tooling, project setup	Auth + layout shell
2 — React Depth	4–6	Fiber internals, state arch, custom hooks	Kanban + command palette
3 — Patterns	7–9	Observer, Command, Strategy, Compound	Notification system + Storybook
4 — Network/Cache	10–13	HTTP, caching, WebSocket, API design	Typed API client + offline mode
5 — Sys Design	14–16	Architecture, performance, testing	90+ Lighthouse + CI/CD
6 — Ship & Hunt	17–20	Launch, resume, interviews, offers	15+ LPA offer

Daily habit: 1 DSA problem (morning) · 1 concept deep-dive · 1–2 hrs building DevFlow · 1 system design sketch per week

PHASE 1 — Foundation & TypeScript Mastery

Weeks 1–3 · Senior engineers write code that is impossible to misuse

TypeScript — compiler level

- Mapped types: { [K in keyof T]: ... }
- Conditional types + infer keyword
- Template literal types for API routes
- Discriminated unions for state machines
- Module augmentation, declaration merging
- Variance: covariance vs contravariance

JS Engine internals

- V8 hidden classes & inline caching
- Call stack, heap, event loop in detail
- Microtask vs macrotask queue — execution order
- Memory leaks in SPAs — closures, listeners
- Garbage collection — mark & sweep
- WeakMap/WeakRef for leak prevention

Senior-grade tooling

- Vite: manual chunks, rollup config, aliases
- ESLint: custom rules, import ordering
- Husky + commitlint + lint-staged
- GitHub Actions: lint, type-check, test, deploy

Project architecture

- Feature-based folder structure
- Barrel exports with re-export control
- Dependency direction rules (no circular)
- Shared vs feature-local code boundaries

DevFlow Features to Build

Feature 1	**Scaffolding + design token system** Vite + React 19 + TS strict mode. Feature folders. Token-based CSS variables. Base components (Button, Input, Badge) with full generic props. CI pipeline on GitHub Actions.
Feature 2	**Auth system — typed state machine** Discriminated union AuthState: Unauthenticated | Authenticating | Authenticated | Error. JWT + refresh token rotation. Route guards via React Router v6 loaders. MSW mocks.
Feature 3	**Layout shell — compound component** Sidebar + topbar as compound components. Collapsible with localStorage. Dark mode via CSS custom properties. Full keyboard nav (a11y). Teaches component composition patterns early.

DSA Focus	
■	Arrays & hashmaps — Two Sum, Group Anagrams, Top K Frequent Elements
■	Strings — Longest Substring Without Repeat, Valid Palindrome, Anagram Check
■	Stack — Valid Parentheses, Min Stack, Daily Temperatures

PHASE 2 — React Internals & State Architecture

Weeks 4–6 · Go past the API surface. Understand how React decides what to re-render

React Fiber & reconciler

- Fiber nodes: type, key, pendingProps, memoizedState
- Work loop: beginWork → completeWork → commit
- Lanes & priority scheduling
- Double buffering: current vs workInProgress tree
- When does React bail out of re-renders?

Concurrent features

- useTransition — non-blocking state updates
- useDeferredValue — stale-while-revalidate UI
- Suspense boundaries + streaming SSR
- React 19: Actions, useOptimistic
- Error boundaries — where to place them

State architecture at scale

- Server state vs client state — clear separation
- React Query: stale time, gc time, invalidation keys
- Zustand: slices, devtools, persist middleware
- Optimistic updates with rollback on error
- Selector memoization — avoid over-subscribing

Custom hooks — advanced

- Stale closure problem — when refs beat state
- useEvent pattern (stable callback identity)
- Composing hooks — single responsibility
- Hook testing with renderHook

DevFlow Features to Build

Feature 4	**Kanban board — drag, drop, undo/redo** Full kanban with @dnd-kit. Zustand for board state with undo/redo via Command pattern (history stack). Optimistic updates via React Query. useTransition for filter animations. Your first design pattern in production.
Feature 5	**Command palette — Cmd+K global search** Portal-rendered command palette. Fuzzy search with fuse.js. useDeferredValue keeps input responsive. Custom useHotkeys hook with keyboard shortcut registry (Observer pattern). Result virtualization.
Feature 6	**Code snippet manager** CRUD with Monaco Editor (syntax highlight). All data via React Query with optimistic adds. Custom useSnippets hook encapsulates all logic. Tag-based filtering. Demonstrates clean hook extraction seniors expect.

DSA Focus	
■	Binary search — Search Rotated Array, Find Peak Element, Koko Eating Bananas
■	Sliding window — Longest Repeating Character, Permutation in String
■	Two pointers — 3Sum, Container With Most Water, Trapping Rain Water

PHASE 3 — Design Patterns in Production

Weeks 7–9 · The biggest differentiator between mid and senior — reach for the right pattern naturally

Patterns to Master — with Real Frontend Implementations

Observer	Pub/sub event bus for cross-feature communication. Custom typed EventEmitter. React context as observable store. Cleanup: unsubscribe in useEffect return. → **DevFlow: real-time notification system**
Command	Encapsulate actions as objects with execute/undo. History stack for undo/redo. Macro commands for batch ops. → **DevFlow: kanban undo/redo + bulk task actions**
Compound Component	Implicit state sharing via context. Flexible composition without prop drilling. Controlled vs uncontrolled variants. → **DevFlow: Modal, Tabs, Dropdown components**
Strategy	Swappable algorithms at runtime. Sort/filter strategies for lists. Different validation rules per form type. → **DevFlow: pluggable task filter/sort engine**
Proxy & Decorator	JS Proxy for reactive objects. API client decorator: logging, retry, auth. HOC as decorator in React. → **DevFlow: typed API client with retry + auth decorator**
Factory	Abstract object creation from consumers. Component factory for dynamic rendering. Widget registry pattern. → **DevFlow: dashboard widget registry**

DevFlow Features to Build

Feature 7	**Real-time notifications — Observer + WebSocket** Custom typed EventEmitter as notification bus. WebSocket hook with reconnect strategy (exponential backoff). Toast system as compound component. Notification bell with unread count.
Feature 8	**Pluggable filter engine — Strategy pattern** Task filters as swappable strategy objects. Filter composition (AND/OR). URL-sync state so filters persist on share. Sorting strategies. A mini query engine showing architectural thinking.
Feature 9	**Storybook component library** Extract all shared components into documented Storybook. Write stories for every state. Deploy to Chromatic. This is your design system — link it in your resume and explain compound component decisions in interviews.

DSA Focus	
■	Trees — Max Depth, Level Order, Lowest Common Ancestor, Serialize/Deserialize
■	BFS/DFS — Number of Islands, Word Ladder, Course Schedule

■	Recursion + backtracking — Subsets, Permutations, N-Queens

PHASE 4 — Networking, APIs & Caching

Weeks 10–13 · The layer most frontend devs treat as a black box. Seniors own it.

What to Learn — Deeply

HTTP deep dive

- HTTP/1.1 vs HTTP/2 vs HTTP/3 — multiplexing, QUIC
- Request lifecycle: DNS → TCP → TLS → HTTP
- Headers: Cache-Control, ETag, If-None-Match, Vary
- CORS — preflight, simple requests, credentials
- Content negotiation, compression (gzip, br)
- HOL blocking, connection keep-alive, pipelining

Caching hierarchy

- Browser cache: memory cache vs disk cache
- Cache-Control: max-age, s-maxage, stale-while-revalidate
- React Query: staleTime, gcTime, cache keys
- Service Worker: cache-first, network-first, SWR strategies
- CDN caching — edge cache, cache busting with hashes
- Cache invalidation — the hardest problem in CS

Real-time protocols

- WebSocket — handshake, frames, ping/pong
- SSE — when to use over WebSocket
- Long polling — fallback pattern
- Reconnect strategies — exponential backoff + jitter
- Message ordering, deduplication, idempotency

API design & patterns

- REST: resource modeling, versioning, HATEOAS
- GraphQL: queries, mutations, N+1 problem
- tRPC — type-safe end-to-end APIs
- API client patterns: interceptors, fetch wrappers
- Rate limiting, retry with backoff, circuit breaker

The Caching Hierarchy — L1 to L5

Layer	Mechanism	Scope	Key Concept
L1 Memory	Browser in-memory	Session only	Fastest — no headers needed, back/fwd nav
L2 Disk	Cache-Control headers	Configurable TTL	ETag + If-None-Match for conditional GETs
L3 Service Worker	Cache Storage API	Offline capable	cache-first / network-first strategies
L4 React Query	In-memory JS cache	App lifetime	staleTime + gcTime + invalidation keys
L5 CDN Edge	CDN servers	Global	s-maxage, Surrogate-Key, content hash filenames

Real-time Protocol Comparison

	WebSocket	Server-Sent Events (SSE)	Long Polling
Direction	Full-duplex	Server → Client only	Server → Client
Reconnect	Manual (write yourself)	Automatic built-in	Manual
Scalability	Harder (sticky sessions)	Easy (HTTP/2)	High server load
Use for	Chat, collab, games	Notifications, feeds	Legacy fallback

DevFlow Features to Build

Feature 10	**Typed API client — Proxy + Decorator pattern** Build a fully-typed HTTP client using JS Proxy. Decorator layers: auth (inject token), retry (exponential backoff), logger, error normalizer. The Proxy + Decorator patterns shipped as one production module.
Feature 11	**Offline mode + service worker cache** Service worker with network-first for API, cache-first for assets. IndexedDB for offline task queue (sync on reconnect). Online/offline indicator. Background sync on reconnect. Shows you understand the full browser stack.
Feature 12	**Analytics dashboard — WebWorker + streaming data** Heavy chart computations offloaded to Web Worker (no jank). Recharts for line, bar, pie. Streaming data via SSE for live metrics. Date range picker. CSV export. Performance at a browser-architecture level.
Feature 13	**Smart caching layer — stale-while-revalidate** Implement stale-while-revalidate at the app level. React Query fine-tuning per resource type (tasks: 30s stale, users: 5min stale). Cache invalidation on mutations. Document decisions in README — this is system design thinking.

DSA Focus	
■	Dynamic programming — Climbing Stairs, Coin Change, Longest Common Subsequence, Edit Distance
■	Heaps / priority queues — Top K Elements, Kth Largest, Find Median from Stream
■	Intervals — Merge Intervals, Meeting Rooms I & II, Insert Interval

PHASE 5 — System Design & Performance Engineering
Weeks 14–16 · Where offers are won and lost. Senior = defensible tradeoffs + systems thinking

Frontend System Design — The Full Playbook

The Answer Framework — Use Every Time

What to cover
Functional (what must it do?) + Non-functional (scale, perf, a11y, offline?). State what is out of scope explicitly. Clarify user count, device targets, latency budget.
Component breakdown — draw the tree. Data flow — where does state live? API contracts — what does frontend need? Rendering strategy — CSR/SSR/SSG —
Caching strategy per resource. Real-time: WebSocket vs SSE vs polling. Performance: Core Web Vitals targets. Error handling + loading states.
What did you choose and why? What did you sacrifice? What would you do at 10x scale? Accessibility, i18n, security.

Must-Practice System Design Questions

Classic	**Design an autocomplete / typeahead component** Debounce, client vs server search, caching recent results, keyboard nav, ARIA, cancel in-flight requests, 1M users.
Classic	**Design a real-time collaborative doc editor** OT vs CRDT, WebSocket architecture, presence indicators, cursor sync, offline edit queue, conflict resolution.
Classic	**Design a design system / component library** Token architecture, semver, breaking change policy, Storybook, a11y guarantees, theming API, tree-shakeable exports.
Advanced	**Design an infinite scroll / virtualized feed** IntersectionObserver vs scroll events, react-virtual windowing, bi-directional scroll, data prefetching, cache eviction.
Advanced	**Design a file upload system** Chunked uploads, resumable uploads (TUS protocol), progress tracking, parallel chunks, retry, client-side validation.
Senior	**Design a micro-frontend platform** Module Federation, shared vs isolated deps, routing, shared auth, cross-MFE communication, independent deployment.

Architecture Topics to Study

Component architecture

- Atomic design: atoms → molecules → organisms → pages
- Feature boundaries — what belongs where
- Shared vs domain-specific components
- Component versioning in a design system

Micro-frontend architecture

- Module Federation with Vite/Webpack
- Shared dependencies strategy
- Routing across micro-frontends
- When NOT to use micro-frontends

Rendering strategies

- CSR vs SSR vs SSG vs ISR — tradeoffs per use case
- Streaming SSR with Suspense
- React Server Components — mental model
- Hydration — full, partial, island architecture

Scale & reliability

- Feature flags — gradual rollout, A/B testing
- Error boundaries + Sentry integration
- Monitoring: real user metrics (RUM)
- Rollback strategies — versioned deployments

DevFlow Features to Build

Feature 14	**Performance engineering sprint** Run Lighthouse. Document before scores. Ship: route-level code splitting, react-virtual for task lists, image lazy loading, font preloading, memoized Zustand selectors, Vite manual chunks. Target: LCP <2s, INP <200ms, 90+ Lighthouse. Write a blog post about the case study.
Feature 15	**Full test suite + CI/CD pipeline** RTL unit tests for all components with MSW mocking. Playwright E2E: login, create task, drag card, search, offline mode. GitHub Actions: lint → typecheck → unit → E2E → deploy to Vercel. 80%+ coverage badge. Preview deploys on every PR.

DSA Focus	
■	Graphs — Dijkstra's, topological sort, detect cycle, word ladder
■	Tries — implement Trie, word search, autocomplete design
■	Bit manipulation — Single Number, Counting Bits, Reverse Bits

PHASE 6 — Ship, Polish & Land the Offer

Weeks 17–20 · DevFlow is live. Resume is a weapon. Offer by week 20.

Ship DevFlow like a Senior Engineer

Production Checklist	Content to Publish
✓ Deployed on Vercel with custom domain ✓ CSP headers, security audit passed ✓ Sentry error tracking integrated ✓ 90+ Lighthouse, WCAG AA accessibility ✓ Storybook deployed to Chromatic ✓ 80%+ test coverage badge on README	✓ README: architecture decisions, tech choices, screenshots ✓ Blog 1: "Real-time kanban with WebSockets & optimistic updates" ✓ Blog 2: "From 54 to 94 — a Lighthouse case study" ✓ Blog 3: "Design patterns I used in production React" ✓ LinkedIn: post 1 engineering insight per week

Resume — Write Like a Senior Engineer

✗ Built kanban board with React and drag and drop	✓ Architected real-time kanban with Command-pattern undo/redo, optimistic updates, and WebSocket sync — zero data loss on reconnect
✗ Improved app performance	✓ Drove Lighthouse score from 54 → 94: route splitting, list virtualization (react-virtual), Web Worker offload for chart computation — LCP dropped from 4.1s to 1.7s
✗ Used design patterns in the project	✓ Implemented Observer, Command, Strategy, and Proxy patterns to decouple feature modules — enabled independent feature shipping without merge conflicts

Where to Apply at 15+ LPA

Company Type	Examples	Range	Strategy
Product startups (B/C)	Razorpay, Groww, CRED, Zepto, Slice	18–30 LPA	Skill > tenure. Apply boldly.
Mid-size product cos	Postman, Chargebee, Freshworks, BrowserStack	15–22 LPA	More structured, good first switch.
Remote / global	Toptal, arc.dev, Deel, Lemon.io, Andela	$40–80K USD	Best ROI — dollar salary in India.
Big tech adjacent	Google, Microsoft, Atlassian India	25–50 LPA	Need strong DSA. Apply in parallel.

DSA Focus	
■	Full revision of all Neetcode Blind 75 — aim for 60+ solved
■	Timed practice: mediums in under 20 min, identify pattern in first 2 min
■	Talk through solution out loud — interviewers evaluate communication equally

RESOURCES & QUICK REFERENCE

Best Learning Resources

Area	Resource	Why
JS/TS Deep	javascript.info + You Don't Know JS (free on GitHub)	Best free JS deep-dive available
React Internals	react.dev + Nadia Makarevich's blog	Fiber, hooks, patterns explained clearly
System Design	greatfrontend.com + frontendinterviewhandbook.com	Frontend-specific design questions
DSA	Neetcode.io roadmap + Leetcode (sorted by frequency)	Structured, pattern-based approach
Performance	web.dev + Harry Roberts's blog + Chrome DevTools docs	Google's own performance guidance
TypeScript	TypeScript Deep Dive (basarat.gitbook.io)	Free, comprehensive, practical
Mock Interviews	Pramp + Exponent	Practice with real people, get feedback
Patterns	Patterns.dev (free book)	React patterns with real examples

The Senior Mindset — What Interviewers Are Actually Testing

Junior: *"I used React Query for data fetching."*	**Senior:** "I chose React Query because we have a mix of server and client state. For server state, I set staleTime based on how frequently each resource changes — tasks at 30s, user profiles at 5 minutes. Cache invalidation happens on mutations. For real-time data like notifications I use WebSocket and bypass the cache layer entirely."
Junior: *"I implemented a design pattern."*	**Senior:** "I reached for the Command pattern for undo/redo because each user action needed to be reversible. Each command is a plain object with execute and undo methods. The history stack is capped at 50 to avoid memory growth. Macro commands batch related operations so users can undo a whole drag-drop sequence as one action."
Junior: *"I improved performance."*	**Senior:** "I profiled with React DevTools and Chrome Lighthouse. The task list was causing layout thrash on every scroll. I switched to react-virtual to only render visible items. Separately, I identified that our analytics computation was blocking the main thread — moved it to a Web Worker. LCP dropped from 4.1s to 1.7s."

> **The core insight:** A junior knows what tools exist. A senior knows why to pick one over another, what it costs, and how it fails. Every topic in this roadmap is framed that way. After 20 weeks, you will not just be able to answer "what is X" — you will be able to say "I chose X over Y because of Z tradeoff, and here is what I would change at 10x scale." That is what 15+ LPA answers sound like.